

Infografía

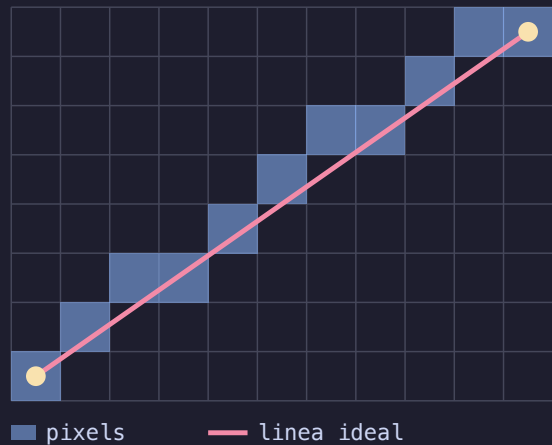
Módulo 3.1 — Rasterización

del píxel al GPU · UPB · Gestión I/2026

El problema

La pantalla es una **grilla discreta de píxeles**.

Una línea matemática $y = m \cdot x + b$ **no vive en la grilla**: pasa por puntos con coordenadas reales.



Para cada paso hay que **decidir** qué pixel encender para *aproximar* la línea ideal.

Distintos algoritmos toman esa decisión de distintas maneras y con distintos compromisos.

Esto es lo que hace el GPU por debajo cuando un shader rasteriza un triángulo.

La progresión del módulo

#	Archivo	Concepto
1	<code>ex1_dda.py</code>	DDA: floats + redondeo
2	<code>ex2_bresenham.py</code>	Bresenham simple (un octante)
3	<code>ex3_bresenham_full.py</code>	Bresenham generalizado (8 octantes)
4	<code>ex4_circle.py</code>	Midpoint circle (<i>ejercicio</i>)
5	<code>ex5_triangle_fill.py</code>	Scanline triangle fill
6	<code>ex6_wu_antialiasing.py</code>	Wu: anti-aliasing

Líneas → curvas → rellenos → AA.

Líneas

DDA y Bresenham

DDA – el algoritmo "obvio"

```
def get_line(x0, y0, x1, y1):  
    dx, dy = x1 - x0, y1 - y0  
    steps = max(abs(dx), abs(dy))  
    x_inc = dx / steps  
    y_inc = dy / steps  
  
    points = []  
    x, y = float(x0), float(y0)  
    for _ in range(steps + 1):  
        points.append((round(x), round(y)))  
        x += x_inc  
        y += y_inc  
    return points
```

Avanzar en floats, redondear al pixel más cercano. Corto, leíble, didáctico.

DDA — por qué no alcanza

```
get_line(0, 0, 10000, 7000)
```

- Cada paso suma `y_inc = 0.7` en un `float`.
- 10 000 pasos → **error de redondeo acumulado**.
- En hardware antiguo (sin FPU), los floats eran **lentos**.

¿se puede hacer todo con enteros?

Bresenham — la idea

Para cada paso en **x**, decidir si **también** subimos en **y** usando una *variable de decisión* **P_k**:

```
def get_line(x0, y0, x1, y1):    # solo dx >= dy >= 0
    dx, dy = x1 - x0, y1 - y0
    xk, yk = x0, y0
    points = [(x0, y0)]
    Pk = 2 * dy - dx
    while xk < x1:
        xk += 1
        if Pk > 0:
            yk += 1
            Pk += 2 * dy - 2 * dx
        else:
            Pk += 2 * dy
        points.append((xk, yk))
    return points
```

*Solo enteros, sin redondeo, sin error acumulado. Pero **un solo octante**.*

¿Y los otros 7 octantes?



El simple solo cubre $dx \geq dy \geq 0$.

Para cubrir los **8 octantes** hay que manejar:

- signo de dx , dy → dirección (sx , sy)
- cuál eje es el "rápido" (el de mayor magnitud)

Bresenham generalizado

```
def get_line(x0, y0, x1, y1):  
    dx, dy = abs(x1 - x0), abs(y1 - y0)  
    sx = 1 if x0 < x1 else -1  
    sy = 1 if y0 < y1 else -1  
    err = dx - dy  
  
    x, y = x0, y0  
    while True:  
        points.append((x, y))  
        if x == x1 and y == y1: break  
        e2 = 2 * err  
        if e2 > -dy: err -= dy; x += sx  
        if e2 < dx: err += dx; y += sy  
    return points
```

Un solo error común; los dos `if` deciden en qué eje (o ambos) avanzar.

Curvas

Midpoint circle

Círculo – la misma idea, otra curva

Misma estrategia que Bresenham: **variable de decisión entera**.

Truco extra: el círculo es **8-simétrico**. Calculamos solo **1/8** y reflejamos:

```
def plot_8_symmetric(xc, yc, x, y):  
    return [  
        (xc + x, yc + y), (xc - x, yc + y),  
        (xc + x, yc - y), (xc - x, yc - y),  
        (xc + y, yc + x), (xc - y, yc + x),  
        (xc + y, yc - x), (xc - y, yc - x),  
    ]
```

8× menos trabajo. Y nunca se desincronizan los píxeles.

Ejercicio — `ex4_circle.py`

```
x, y = 0, r
p = 1 - r          # variable de decision
while x < y:
    x += 1
    if p < 0:
        p += 2*x + 1
    else:
        y -= 1
        p += 2*(x - y) + 1
    # graficar los 8 puntos simetricos
```

Completar el bucle. `plot_8_symmetric` ya está.

Extensiones: círculos concéntricos · comparar contra `sqrt(r2 - x2)` · elipse · círculo relleno (líneas horizontales entre pares simétricos).

Relleno

Scanline triangle fill

De 1D a 2D

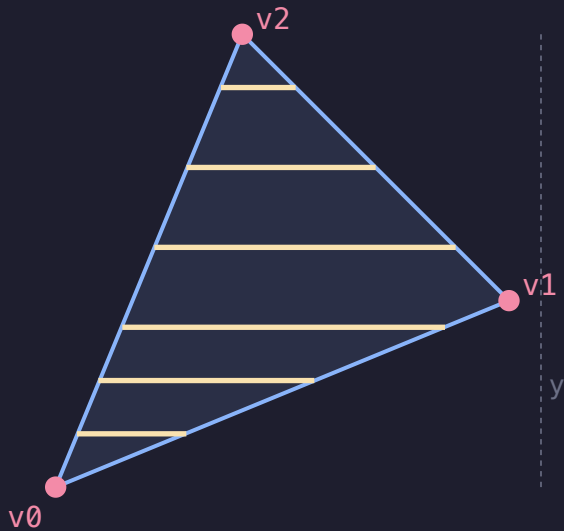
Hasta acá: rasterizar **líneas y curvas** (1D dentro de un plano 2D).

Ahora: **regiones**. La primitiva 2D fundamental → el **triángulo**.

Esto es la base del rasterizer de TODAS las GPUs modernas.

Todo en 3D se descompone en triángulos. Pintar un triángulo en pantalla = scanline fill.

Scanline – la idea



1. Ordenar los 3 vértices por **y**.
2. Partir en dos mitades (panza inferior + superior).
3. Para cada **scanline horizontal y**:
 - encontrar las **x** de las dos aristas activas
 - rellenar la franja entre ellas

El código de la mitad inferior

```
v0, v1, v2 = sorted([p0, p1, p2], key=lambda p: p[1])
(x0, y0), (x1, y1), (x2, y2) = v0, v1, v2

def edge_x(ya, yb, xa, xb, y):
    if yb == ya: return float(xa)
    t = (y - ya) / (yb - ya)
    return xa + (xb - xa) * t

# mitad inferior: aristas v0->v1 y v0->v2
for y in range(y0, y1):
    xa = edge_x(y0, y2, x0, x2, y)
    xb = edge_x(y0, y1, x0, x1, y)
    for x in range(int(min(xa, xb)), int(max(xa, xb)) + 1):
        pixels.append((x, y))
```

La mitad superior es lo mismo cambiando las aristas activas.

Los casos degenerados importan

- Triángulos **casi-horizontales** (`y1 ≈ y0`).
- Vértices **repetidos** o colineales.
- Aristas verticales (`yb == ya`).

No son edge cases académicos — son el 95% de los bugs de rasterización.

Por eso `edge_x` chequea `if yb == ya` antes de dividir.

Anti-aliasing

algoritmo de Wu

El supuesto que rompe Wu

Hasta acá: **un pixel está prendido o apagado** (binario).

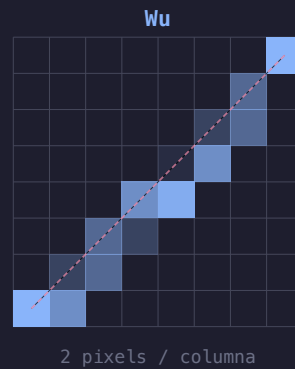
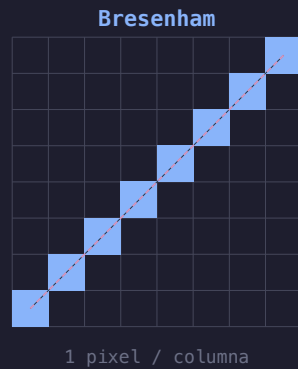
La línea queda escalonada — el "aliasing".

¿y si el pixel tiene intensidad fraccional?

Wu: **dos píxeles por columna**, con intensidades que suman 1, proporcionales a qué tan cerca está la línea ideal.

El ojo los integra → el escalón desaparece visualmente.

Bresenham vs. Wu



```
intery = y0 + gradient
for x in range(x0 + 1, x1):
    y_int = int(intery)
    frac = intery - y_int
    plot(x, y_int,      1 - frac)
    plot(x, y_int + 1,  frac)
    intery += gradient
```

Devuelve `(x, y, intensity)` con
`intensity ∈ [0, 1]`.

La idea grande

Estamos **convolucionando implícitamente** la línea ideal con el "kernel" del pixel.

- El pixel deja de ser una celda binaria → pasa a ser un área con cobertura.
- La intensidad = cuánta área del pixel cubre la línea.

Esta idea —*coverage / cobertura*— vuelve más adelante:

- multisample AA en el GPU
- filtros de textura
- alpha blending

Patrón del módulo

Función pura + visualizador

Cada `get_*` devuelve `list[tuple]` y **no toca arcade**:

```
from ex3_bresenham_full import get_line

points = get_line(0, 0, 10, 5)
assert (0, 0) in points
assert (10, 5) in points
```

El algoritmo es testeable sin pintar nada.

`visualize.py` (compartido) toma la lista y dibuja la grilla. Permite **separar la lógica del render** — un patrón que se repite a lo largo del curso.

¿Por qué aprender esto si la GPU lo hace?

1. **Es lo que hace la GPU por dentro.** Saberlo cambia cómo lees un shader.
2. Aparece de nuevo en:
 - shaders de fragmento (`6._transformations`, `7._godot/shaders`)
 - filtros, blur, sampling
 - cálculos de cobertura y AA
3. **Es el primer puente entre matemáticas continuas y pixeles discretos** — el tema central de gráficos por computadora.

¿Preguntas?

Próxima clase — Sprites y assets

```
uv run 3._arcade/3.1_algorithms/ex3_bresenham_full.py
```